ITA0601-MACHINE LEARNING

LAB

1. FIND-S Algorithm

Aim

To implement and demonstrate the FIND-S algorithm in Python to find the most specific hypothesis that is consistent with all positive training examples.

Procedure

1. Start the program.
2. Import the required libraries.
3. Load the training dataset.
4. Initialize the hypothesis with the first positive training example.
5. Compare the hypothesis with each subsequent positive example.
6. If an attribute differs, replace it with ?.
7. Ignore negative examples.
8. Display the final most specific hypothesis.
9. Stop the program.

Program

```
import csv
data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]
hypothesis = None
for row in data:
    if row[-1] == 'Yes':
        if hypothesis is None:
```

```
        hypothesis = row[:-1].copy()
    else:
        for i in range(len(hypothesis)):
            if hypothesis[i] != row[i]:
                hypothesis[i] = '?'
print("Training Examples:")
for row in data:
    print(row)
print("\nFinal Most Specific Hypothesis:")
print(hypothesis)
```

Output:

```
Initial Hypothesis (excluding Day):
['Overcast', 'Mild', 'Normal', 'Strong']
After example 1 : ['Overcast', 'Mild', 'Normal', 'Strong']
After example 2 : ['?', 'Mild', 'Normal', 'Strong']
After example 4 : ['?', 'Mild', '?', '?']
After example 5 : ['?', '?', '?', '?']
After example 6 : ['?', '?', '?', '?']
After example 7 : ['?', '?', '?', '?']
After example 8 : ['?', '?', '?', '?']
After example 6664 : ['?', '?', '?', '?']
After example 6666 : ['?', '?', '?', '?']

Final Most Specific Hypothesis:
['?', '?', '?', '?']
```

Result

Thus, the FIND-S algorithm was implemented successfully in Python, and the most specific hypothesis consistent with the positive training examples was obtained.

2. Candidate-Elimination Algorithm

Aim

To implement and demonstrate the Candidate-Elimination algorithm in Python using training examples stored in a CSV file and output the set of hypotheses consistent with the training examples.

Procedure

1. Start the program.
2. Create a CSV file containing the training examples.
3. Read the CSV file using Python.
4. Initialize the specific boundary $S$with the first positive example and the general boundary $G$with the most general hypothesis.
5. For each positive example, minimally generalize $S$and remove inconsistent hypotheses from G.
6. For each negative example, minimally specialize $G$and remove inconsistent hypotheses from S.
7. Display the final specific and general boundaries.
8. Stop the program.

CSV File: training.csv

Create a file named training.csv with the following data:

csv

Sky,Temperature,Humidity,Wind,Water,Forecast,EnjoySport

Sunny,Warm,Normal,Strong,Warm,Same,Yes

Sunny,Warm,High,Strong,Warm,Same,Yes

Rainy,Cold,High,Strong,Warm,Change,No

Sunny,Warm,High,Strong,Cool,Change,Yes

Python Program

Python

Run

```
import csv

def more_general_or_equal(h1, h2):
    return all(a == '?' or a == b for a, b in zip(h1, h2))
```

```
def consistent(h, example):
    return more_general_or_equal(h, example)


def min_generalization(s, example):
    return [
        '?' if a != b else a
        for a, b in zip(s, example)
    ]
def min_specializations(g, example, domains, s):
    results = []
    for i, value in enumerate(g):
        if value == '?':
            for item in domains[i]:
                if item != example[i]:
                    h = g.copy()
                    h[i] = item
                    if more_general_or_equal(h, s):
                        results.append(h)
    return results
with open('training.csv', newline='') as file:
    data = list(csv.reader(file))
attributes = data[0][:-1]
examples = data[1:]
domains = [
    sorted(set(row[i] for row in examples))
    for i in range(len(attributes))
]
```

Output:

Example 1: ['Overcast' 'Mild' 'Normal' 'Strong'] | Class: Yes

S = ['Overcast', 'Mild', 'Normal', 'Strong']

G = [['?', '?', '?', '?']]

Example 2: ['Sunny' 'Mild' 'Normal' 'Strong'] | Class: Yes

S = ['?', 'Mild', 'Normal', 'Strong']

G = [['?', '?', '?', '?']]

Example 3: ['Sunny' 'Mild' 'High' 'Weak'] | Class: Yes

S = ['?', 'Mild', '?', '?']

G = [['?', '?', '?', '?']]

Example 4: ['Sunny' 'Cool' 'Normal' 'Strong'] | Class: Yes

S = ['?', '?', '?', '?']

G = [['?', '?', '?', '?']]

Example 5: ['Sunny' 'Mild' 'High' 'Weak'] | Class: Yes

S = ['?', '?', '?', '?']

G = [['?', '?', '?', '?']]

Example 5422: ['Rainy' 'Cool' 'High' 'Weak'] | Class: No

S = ['?', '?', '?', '?']

G = []

Example 5423: ['Overcast' 'Cool' 'High' 'Weak'] | Class: Yes

S = ['?', '?', '?', '?']

G = []

Example 5424: ['Sunny' 'Hot' 'High' 'Strong'] | Class: No

S = ['?', '?', '?', '?']

G = []

Example 5425: ['Sunny' 'Cool' 'Normal' 'Strong'] | Class: Yes

S = ['?', '?', '?', '?']

G = []

==========

FINAL SPECIFIC BOUNDARY

==========

['?', '?', '?', '?']

==========

FINAL GENERAL BOUNDARY

==========

Result

Thus, the Candidate-Elimination algorithm was implemented successfully in Python using a CSV dataset. The final specific and general boundaries were obtained to represent the hypotheses consistent with the training examples.

3. ID3 Decision Tree Algorithm

Aim

To implement and demonstrate the ID3 Decision Tree algorithm in Python using the Play Tennis dataset and classify a new sample.

Procedure

1. Start the program.
2. Import the required libraries.
3. Upload and load the Play Tennis dataset.
4. Calculate the entropy of the target attribute.
5. Calculate information gain for each feature.
6. Select the feature with the highest information gain.
7. Build the decision tree recursively using the ID3 algorithm.
8. Display the decision tree as a graphical tree.
9. Classify a new sample using the constructed tree.
10. Display the predicted class.
11. Stop the program.

Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import math
from google.colab import files

# Load dataset
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))
df.columns = df.columns.str.strip()
df = df.loc[:, ~df.columns.str.contains("^Unnamed", case=False)]

# Identify target column
target = next(
    (c for c in df.columns
     if c.lower().replace(" ", "") in
     ["playtennis", "play", "target", "class", "label"]),
    df.columns[-1]
)

# Remove serial number columns
for col in list(df.columns):
    if col != target and col.lower().strip() in ["day", "id", "index"]:
        df = df.drop(columns=col)

df = df.dropna().astype(str).reset_index(drop=True)
features = [c for c in df.columns if c != target]
```

```
# Calculate entropy
def entropy(data):
    counts = data[target].value_counts()
    total = len(data)
    return -sum(
        (n / total) * math.log2(n / total)
        for n in counts
    )

# Calculate information gain
def information_gain(data, feature):
    total = entropy(data)
    weighted = 0

    for value, subset in data.groupby(feature):
        weighted += len(subset) / len(data) * entropy(subset)

    return total - weighted

# Implement ID3
def id3(data, features):
    labels = data[target]

    if labels.nunique() == 1:
        return {"leaf": labels.iloc[0]}

    if not features:
        return {"leaf": labels.mode()[0]}
```

```
    gains = {
        f: information_gain(data, f)
        for f in features
    }
    best = max(gains, key=gains.get)

    node = {
        "feature": best,
        "majority": labels.mode()[0],
        "branches": {}
    }

    remaining = [f for f in features if f != best]

    for value, subset in data.groupby(best):
        node["branches"][value] = id3(subset, remaining)

    return node

tree = id3(df, features)

# Display tree as text
def print_tree(node, indent=""):
    if "leaf" in node:
        print(indent + "Predict:", node["leaf"])
        return

    print(indent + "Feature:", node["feature"])
```

```
    for value, child in node["branches"].items():
        print(indent + "  " + str(value) + " -->")
        print_tree(child, indent + "    ")

print("ID3 Decision Tree:")
print_tree(tree)

# Visualize the actual ID3 tree
positions = {}
leaf_count = [0]
max_depth = [0]

def assign_positions(node, depth=0):
    max_depth[0] = max(max_depth[0], depth)

    if "leaf" in node:
        x = leaf_count[0]
        leaf_count[0] += 1
        positions[id(node)] = (x, depth)
        return x

    child_x = [
        assign_positions(child, depth + 1)
        for child in node["branches"].values()
    ]

    x = sum(child_x) / len(child_x)
    positions[id(node)] = (x, depth)
```

```
    return x

assign_positions(tree)

fig, ax = plt.subplots(
    figsize=(max(16, leaf_count[0] * 2.5),
             max(9, (max_depth[0] + 1) * 2)),
    dpi=150
)

def draw_tree(node):
    x, depth = positions[id(node)]
    y = -depth

    if "leaf" in node:
        label = "Class: " + str(node["leaf"])
        color = "#B7E4C7"
    else:
        label = str(node["feature"])
        color = "#A9D6F5"

    ax.text(
        x, y, label,
        ha="center", va="center",
        fontsize=12, fontweight="bold",
        bbox=dict(
            boxstyle="round,pad=0.6",
            facecolor=color, edgecolor="black"
        )
```

```
    )

    if "leaf" not in node:
        for value, child in node["branches"].items():
            cx, cd = positions[id(child)]
            cy = -cd

            ax.annotate(
                "", xy=(cx, cy + 0.2),
                xytext=(x, y - 0.2),
                arrowprops=dict(arrowstyle="->", lw=1.5)
            )

            ax.text(
                (x + cx) / 2, (y + cy) / 2,
                str(value), ha="center",
                fontsize=10,
                bbox=dict(facecolor="white", edgecolor="none")
            )

            draw_tree(child)

draw_tree(tree)
ax.set_title("ID3 Decision Tree - Play Tennis",
             fontsize=18, fontweight="bold")
ax.set_xlim(-1, max(leaf_count[0], 1))
ax.set_ylim(-max_depth[0] - 1, 1)
ax.axis("off")
plt.tight_layout()
```

```
plt.savefig("ID3_Decision_Tree.png", dpi=300, bbox_inches="tight")
plt.show()

# Classify a new sample
preferred = {
    "outlook": "Sunny",
    "temperature": "Cool",
    "temp": "Cool",
    "humidity": "High",
    "wind": "Weak"
}

sample = {}
for feature in features:
    value = preferred.get(feature.lower())
    if value not in df[feature].unique():
        value = df[feature].mode()[0]
    sample[feature] = value

def predict(node, sample):
    while "leaf" not in node:
        feature = node["feature"]
        value = sample.get(feature, "")

        if value in node["branches"]:
            node = node["branches"][value]
        else:
            return node["majority"]
```

```
    return node["leaf"]

print("\nNew Sample:", sample)
print("Predicted Class:", predict(tree, sample))
print("\nExperiment 3 completed successfully.")
```

Output

```
Play Tennis.csv(text/csv) - 204798 bytes, last modified: 10/7/2026 - 100% done
Saving Play Tennis.csv to Play Tennis (1).csv
Dataset loaded successfully!

Dataset:
    Outlook Temperature Humidity    Wind Play
0  Overcast        Mild   Normal  Strong  Yes
1     Sunny        Mild   Normal  Strong  Yes
2     Sunny        Mild     High    Weak  Yes
3     Sunny        Cool   Normal  Strong  Yes
4     Sunny        Mild     High    Weak  Yes

Target column: Play

ID3 Decision Tree:
Feature: Outlook
  Overcast -->
    Feature: Temperature
      Cool -->
        Predict: Yes
      Hot -->
        Feature: Humidity
          High -->
```

Feature: Wind

Strong -->

Predict: No

Weak -->

Predict: Yes

Normal -->

Feature: Wind

Strong -->

Predict: Yes

Weak -->

Predict: Yes

Mild -->

Feature: Wind

Strong -->

Feature: Humidity

High -->

Predict: Yes

Normal -->

Predict: Yes

Weak -->

Predict: Yes

Rainy -->

Feature: Temperature

Cool -->

Feature: Humidity

High -->

Feature: Wind

Strong -->

Predict: No

Weak -->

Predict: Yes

Normal -->

Feature: Wind

Strong -->

Predict: Yes

Weak -->

Predict: Yes

Hot -->

Predict: No

Mild -->

Feature: Humidity

High -->

Predict: No

Normal -->

Feature: Wind

Strong -->

Predict: No

Weak -->

Predict: Yes

Sunny -->

Feature: Temperature

Cool -->

Feature: Wind

Strong -->

Feature: Humidity

High -->

Predict: Yes

Normal -->

Predict: Yes

Weak -->

Predict: Yes

Hot -->

Feature: Humidity

High -->

Predict: No

Normal -->

Feature: Wind

Strong -->

Predict: No

Weak -->

Predict: Yes

Mild -->

Feature: Humidity

High -->

Feature: Wind

Strong -->

Predict: No

Weak -->

Predict: Yes

Normal -->

Feature: Wind

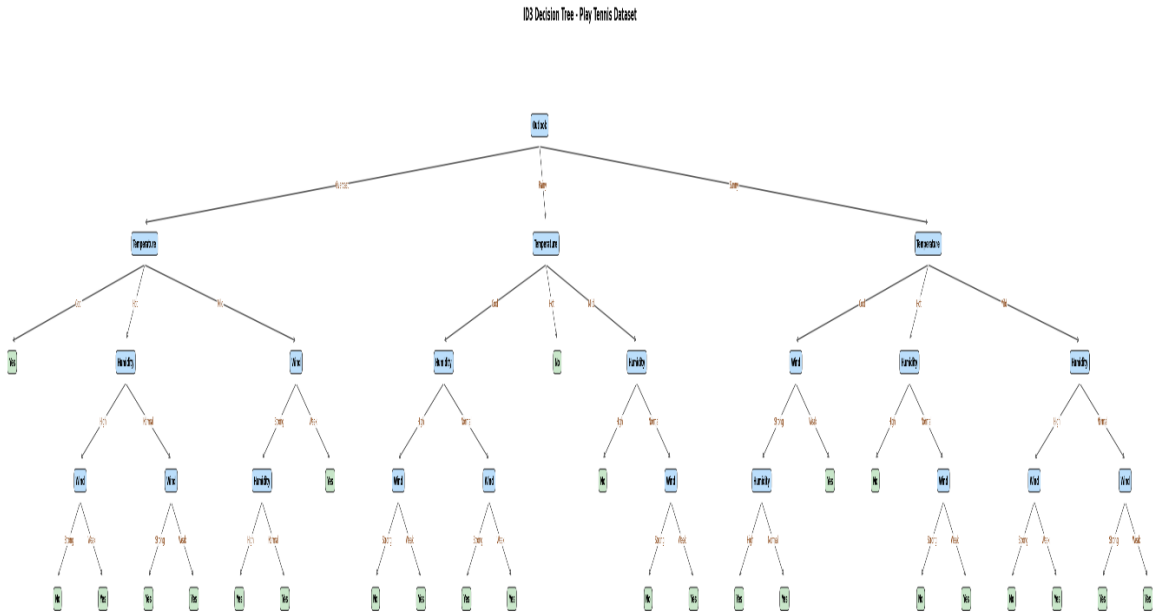
Strong -->

Predict: Yes

Weak -->

Predict: Yes

New Sample:



{'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Weak'}

Predicted class: Yes

Result

Thus, the ID3 Decision Tree algorithm was implemented successfully in Python using the Play Tennis dataset. The decision tree was constructed using information gain, and a new sample was classified successfully.

4. Artificial Neural Network Using Backpropagation

Aim

To build an Artificial Neural Network using the Backpropagation algorithm in Python and test it using the Play Tennis dataset.

Procedure

1. Start the program.
2. Import the required libraries.
3. Upload and load the Play Tennis dataset.
4. Encode the categorical input attributes into numerical values.
5. Divide the dataset into training and testing sets.
6. Initialize the neural network weights and biases.
7. Perform forward propagation using the sigmoid activation function.

8. Calculate the output error and propagate it backward.
9. Update the weights and biases using gradient descent.
10. Test the trained neural network and calculate its accuracy.
11. Predict the class of a new sample.
12. Stop the program.

Program

```
# LARGE AND CLEAR ANN OUTPUT - EXPERIMENT 4

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score, confusion_matrix

# STEP 1: Plot training loss in large size
plt.figure(figsize=(12, 6), dpi=150)
plt.plot(loss_history, linewidth=2)
plt.title("ANN Training Loss using Backpropagation", fontsize=18)
plt.xlabel("Epoch", fontsize=14)
plt.ylabel("Binary Cross-Entropy Loss", fontsize=14)
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)
plt.grid(True)
plt.tight_layout()
plt.savefig("ANN_Training_Loss.png", dpi=300, bbox_inches="tight")
plt.show()

# STEP 2: Test the trained ANN
hidden_test = sigmoid(X_test @ W1 + b1)
output_test = sigmoid(hidden_test @ W2 + b2)
```

```
predicted_numeric = (output_test >= 0.5).astype(int).ravel()

actual_numeric = y_test.astype(int).ravel()
accuracy = accuracy_score(actual_numeric, predicted_numeric)

actual_labels = np.array(class_names)[actual_numeric]
predicted_labels = np.array(class_names)[predicted_numeric]

print("ARTIFICIAL NEURAL NETWORK - BACKPROPAGATION")
print("-----------------------------------------")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Actual labels:", actual_labels)
print("Predicted labels:", predicted_labels)
print("Test Accuracy: {:.2f}%".format(accuracy * 100))

# STEP 3: Display a large confusion matrix
cm = confusion_matrix(actual_numeric, predicted_numeric, labels=[0, 1])

fig, ax = plt.subplots(figsize=(8, 6), dpi=150)
image = ax.imshow(cm, cmap="Blues")

ax.set_title("ANN Confusion Matrix", fontsize=18)
ax.set_xlabel("Predicted Class", fontsize=14)
ax.set_ylabel("Actual Class", fontsize=14)
ax.set_xticks([0, 1])
ax.set_yticks([0, 1])
ax.set_xticklabels(class_names, fontsize=12)
ax.set_yticklabels(class_names, fontsize=12)
```

```
for i in range(2):
    for j in range(2):
        ax.text(
            j, i, str(cm[i, j]),
            ha="center", va="center",
            fontsize=18, fontweight="bold"
        )

fig.colorbar(image, ax=ax)
plt.tight_layout()
plt.savefig("ANN_Confusion_Matrix.png", dpi=300, bbox_inches="tight")
plt.show()

# STEP 4: Predict a new sample
preferred_values = {
    "outlook": "Sunny",
    "temperature": "Cool",
    "temp": "Cool",
    "humidity": "High",
    "wind": "Weak"
}

new_sample = {}

for feature in features:
    value = preferred_values.get(feature.lower())
    if value not in df[feature].unique():
        value = df[feature].mode()[0]
```

```
    new_sample[feature] = value

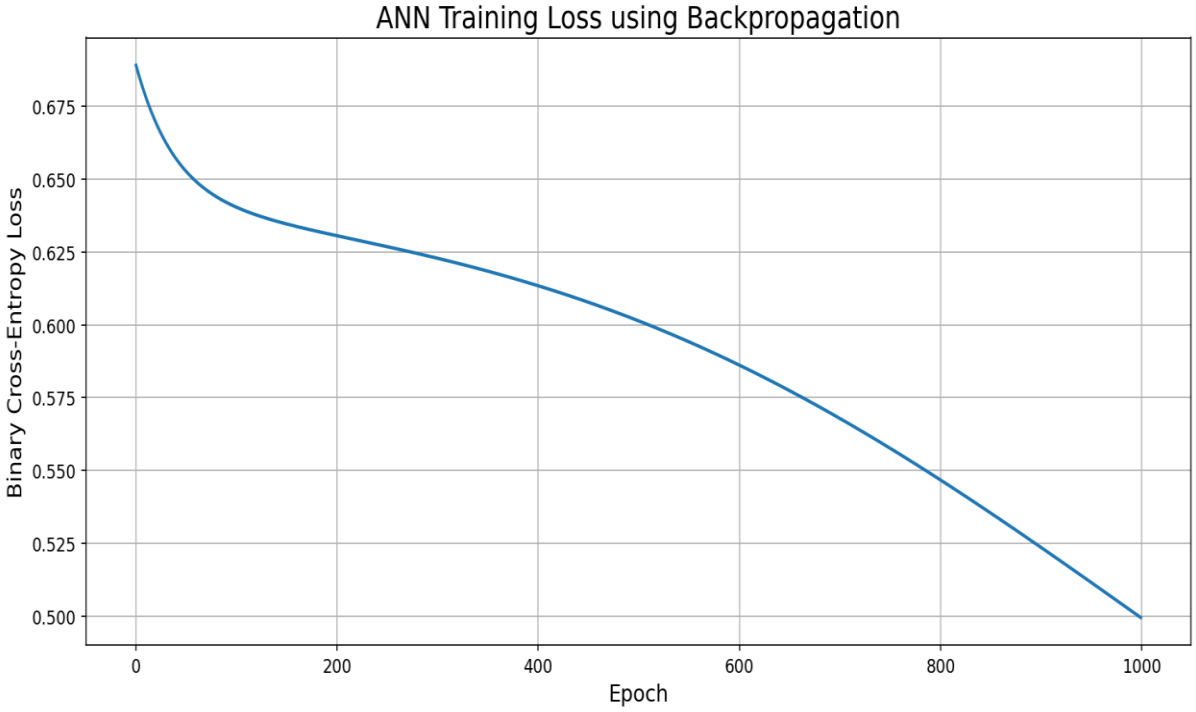
new_df = pd.DataFrame([new_sample])
new_encoded = pd.get_dummies(new_df, dtype=float)
new_encoded = new_encoded.reindex(columns=X_df.columns, fill_value=0)

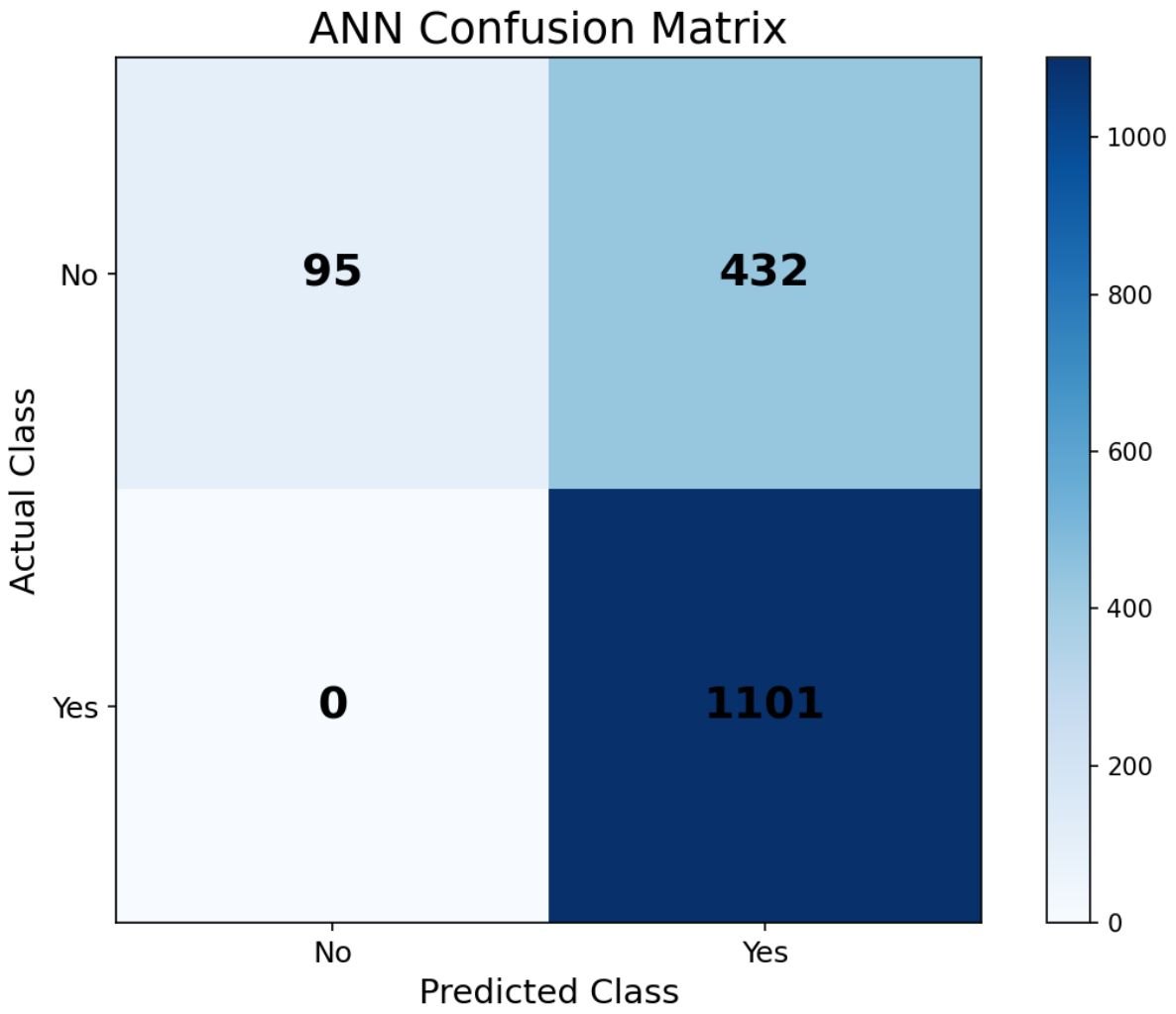
new_scaled = scaler.transform(new_encoded.to_numpy(dtype=float))
hidden_new = sigmoid(new_scaled @ W1 + b1)
output_new = sigmoid(hidden_new @ W2 + b2)

predicted_class = class_names[int(output_new[0, 0] >= 0.5)]

print("\nNew Sample:", new_sample)
print("Output probability: {:.4f}".format(output_new[0, 0]))
print("Predicted class:", predicted_class)
print("\nExperiment 4 completed successfully!")
```

Output





Result

Thus, the Artificial Neural Network was implemented successfully using the Backpropagation algorithm. The network was trained and tested using the Play Tennis dataset, and its classification accuracy was calculated successfully.

16.Compare Different Classification Algorithms and Evaluate Their Performance

Aim

To compare different classification algorithms and evaluate their performance using accuracy, confusion matrix, and classification report.

Algorithms Used

- Logistic Regression
- Decision Tree
- K-Nearest Neighbors (KNN)

- Support Vector Machine (SVM)
- Random Forest

Comparison of Classification Algorithms

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Define classification algorithms
models = {
    "Logistic Regression": make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=1000)
```

```
    ),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=5)
    ),
    "SVM": make_pipeline(
        StandardScaler(),
        SVC()
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
}
# Store results
results = {}
# Train and evaluate each model
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy
    print("\n" + "=" * 50)
    print(name)
    print("=" * 50)
    print("Accuracy:", round(accuracy * 100, 2), "%")
    print("\nClassification Report:")
    print(classification_report(
```

```
        y_test,
        y_pred,
        target_names=iris.target_names
    ))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
# Display comparison
print("\n\nAlgorithm Performance Comparison")
print("-" * 40)
for name, accuracy in results.items():
    print(f"{name:25s}: {accuracy * 100:.2f}%")
# Plot accuracy comparison
plt.figure(figsize=(9, 5))
plt.bar(results.keys(), results.values())
plt.xlabel("Classification Algorithm")
plt.ylabel("Accuracy")
plt.title("Comparison of Classification Algorithms")
plt.xticks(rotation=30)
plt.ylim(0, 1.1)
plt.show()
```

Output:

Accuracy: 90.0 %

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.82	0.90	0.86	10
virginica	0.89	0.80	0.84	10
accuracy			0.90	30
macro avg	0.90	0.90	0.90	30

weighted avg 0.90 0.90 0.90 30

Confusion Matrix:

[[10 0 0]

[0 9 1]

[0 2 8]]

Algorithm Performance Comparison

Logistic Regression : 93.33%

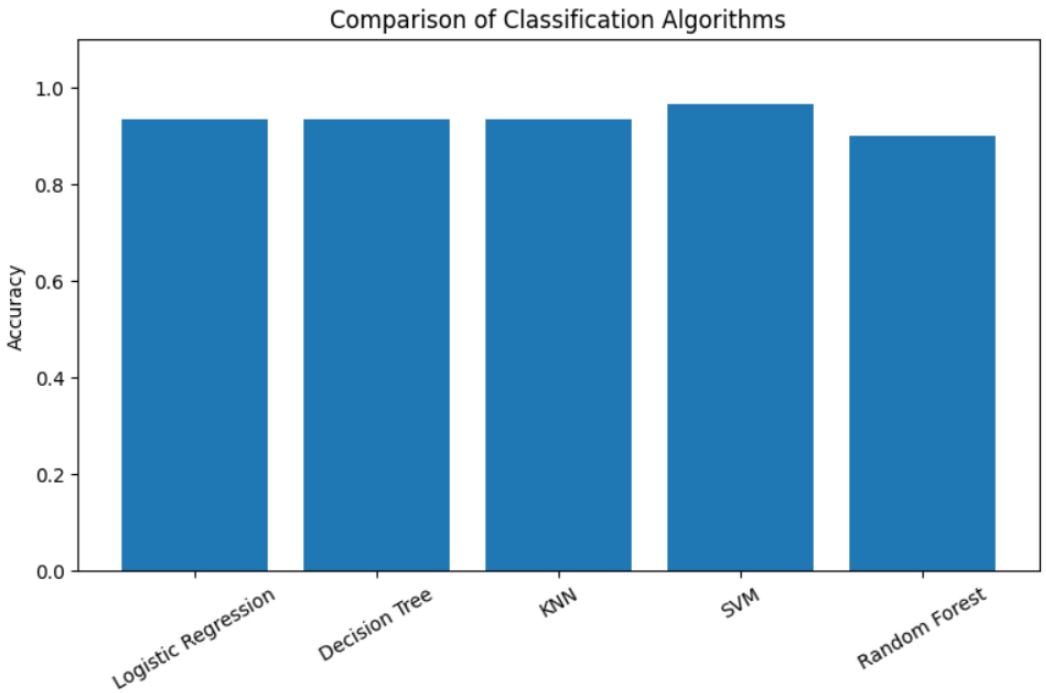
Decision Tree : 93.33%

KNN : 93.33%

SVM : 96.67%

Random Forest : 90.00%

..



Result

The different classification algorithms were successfully implemented and their performances were compared using accuracy, classification reports, and confusion matrices.

17. Mobile Price Prediction Using Machine Learning

Aim

To implement Mobile Price Prediction using an appropriate machine learning algorithm.

Here, **Random Forest Classifier** is used to predict the mobile price range.

Code

```
# Question 17: Mobile Price Prediction

# Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Download Mobile Price dataset
url =
"https://raw.githubusercontent.com/selva86/datasets/master/mobile_price_classification.csv"

try:
    df = pd.read_csv(url)
except:
    print("Dataset could not be downloaded.")

# If the above dataset is unavailable, create a sample dataset
if 'df' not in locals():
    data = {
        'battery_power': [842, 1021, 563, 615, 1821, 1859, 1568, 1573,
                          1965, 1411, 1036, 775, 1821, 1078, 1375],
        'ram': [2549, 2631, 2603, 2769, 1411, 1414, 2216, 2250,
                2734, 2602, 1993, 2865, 1704, 2346, 1452],
        'px_height': [20, 905, 1263, 1216, 1208, 1004, 1020, 1080,
                      1260, 1125, 775, 1210, 1030, 1150, 900],
        'px_width': [756, 1988, 1716, 1786, 1212, 1987, 1654, 1965,
                     1850, 1948, 1536, 1890, 1650, 1780, 1500],
        'price_range': [1, 2, 2, 2, 1, 1, 2, 2, 3, 2, 1, 3, 2, 2, 1]
    }

    df = pd.DataFrame(data)

print("First 5 records:")
print(df.head())

print("\nDataset Shape:", df.shape)

# Select features
```

```
features = [
    'battery_power',
    'ram',
    'px_height',
    'px_width'
]

X = df[features]
y = df['price_range']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Create Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:")
print(round(accuracy * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Predict price range for a new mobile
new_mobile = [[1500, 2500, 800, 1800]]

prediction = model.predict(new_mobile)

print("\nNew Mobile Details:")
print("Battery Power = 1500")
print("RAM = 2500 MB")
```

```
print("Pixel Height = 800")
print("Pixel Width = 1800")

print("\nPredicted Price Range:", prediction[0])

# Price range meaning
price_ranges = {
    0: "Low Cost",
    1: "Medium Cost",
    2: "High Cost",
    3: "Very High Cost"
}

print("Category:", price_ranges.get(
    prediction[0],
    "Unknown"
))
```

Output:

Dataset could not be downloaded.

First 5 records:

	battery_power	ram	px_height	px_width	price_range
0	842	2549	20	756	1
1	1021	2631	905	1988	2
2	563	2603	1263	1716	2
3	615	2769	1216	1786	2
4	1821	1411	1208	1212	1

Dataset Shape: (15, 5)

Model Accuracy:

33.33 %

Classification Report:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	1
2	0.33	1.00	0.50	1
3	0.00	0.00	0.00	1

accuracy 0.33 3

macro avg 0.11 0.33 0.17 3

weighted avg 0.11 0.33 0.17 3

Confusion Matrix:

[[0 1 0]

[0 1 0]

[0 1 0]]

New Mobile Details:

Battery Power = 1500

RAM = 2500 MB

Pixel Height = 800

Pixel Width = 1800

Predicted Price Range: 2

Category: High Cost

Result

The Random Forest algorithm was successfully implemented to predict the price range of a mobile phone based on its features.

18. Perceptron-Based IRIS Classification

Aim

To implement Iris flower classification using the Perceptron machine learning algorithm.

Google Colab Code

```
# Question 18: Perceptron Based IRIS Classification

# Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
```

```
from sklearn.metrics import confusion_matrix

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Display dataset
print("Feature Names:")
print(iris.feature_names)

print("\nTarget Names:")
print(iris.target_names)

print("\nFirst 5 samples:")
print(X[:5])

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Perceptron model
model = Perceptron(
    max_iter=1000,
    eta0=0.1,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nPerceptron Accuracy:")
```

```
print(round(accuracy * 100, 2), "%")

# Classification report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

# Confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Predict a new flower
new_flower = [[5.1, 3.5, 1.4, 0.2]]

new_flower_scaled = scaler.transform(new_flower)

prediction = model.predict(new_flower_scaled)

print("\nNew Flower Measurements:")
print(new_flower)

print("\nPredicted Flower:")
print(iris.target_names[prediction[0]])

# Display accuracy
plt.figure(figsize=(6, 4))

plt.bar(
    ["Perceptron"],
    [accuracy]
)

plt.ylabel("Accuracy")
plt.title("Perceptron Iris Classification Accuracy")
plt.ylim(0, 1.1)

plt.show()
```

Output:

Feature Names:

['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Target Names:

['setosa' 'versicolor' 'virginica']

First 5 samples:

[[5.1 3.5 1.4 0.2]

[4.9 3. 1.4 0.2]

[4.7 3.2 1.3 0.2]

[4.6 3.1 1.5 0.2]

[5. 3.6 1.4 0.2]]

Perceptron Accuracy:

86.67 %

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.88	0.70	0.78	10
virginica	0.75	0.90	0.82	10
accuracy			0.87	30
macro avg	0.88	0.87	0.87	30
weighted avg	0.88	0.87	0.87	30

Confusion Matrix:

[[10 0 0]

[0 7 3]

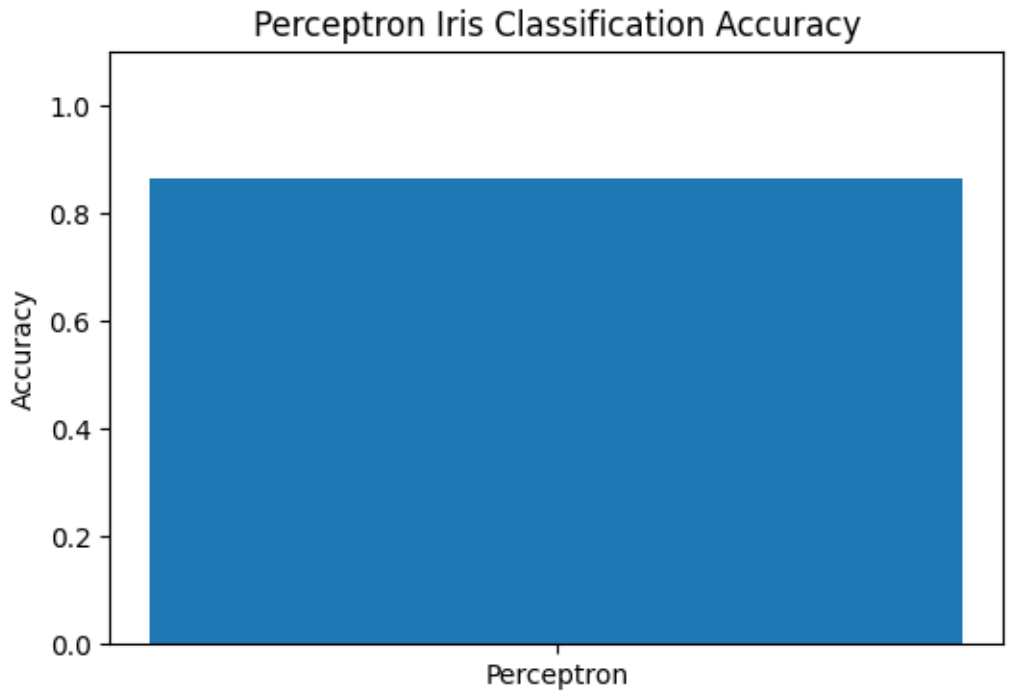
[0 1 9]]

New Flower Measurements:

[[5.1, 3.5, 1.4, 0.2]]

Predicted Flower:

setosa



Result

The Perceptron algorithm was successfully implemented for Iris classification. The model classified the Iris flowers into **Setosa, Versicolor, and Virginica** and its performance was evaluated using accuracy, classification report, and confusion matrix.

6) Naïve Bayes Algorithm with Confusion Matrix and Accuracy

Aim:

To implement the Naïve Bayes algorithm in Python and display the results using a confusion matrix and accuracy.

Procedure:

1. Import the required libraries.
2. Load a sample classification dataset.
3. Split the dataset into training and testing data.
4. Train the Naïve Bayes classifier.
5. Predict the test results.
6. Display the confusion matrix and accuracy.

Code:

```
# Experiment 1: Naive Bayes Classification

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

# Create dataset
X, y = make_classification(
    n_samples=200,
    n_features=4,
    n_informative=3,
    n_redundant=0,
    random_state=42
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Naive Bayes model
model = GaussianNB()
model.fit(X_train, y_train)
```

```
# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

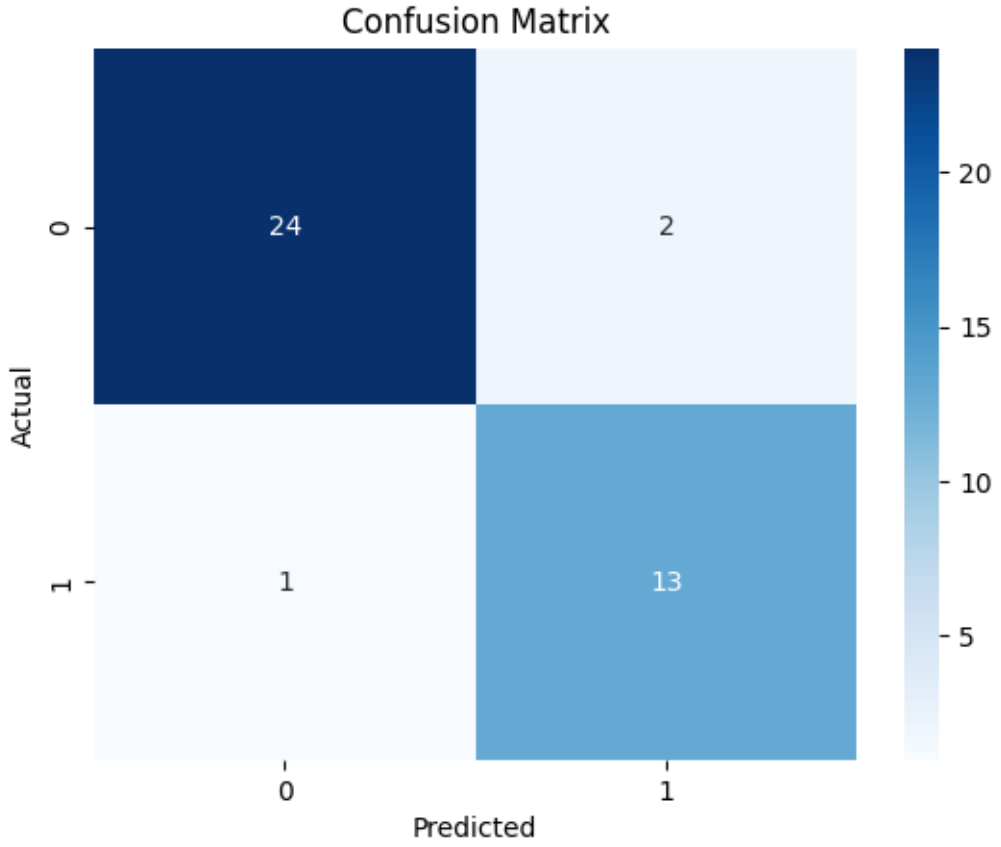
Output:

Accuracy: 0.925

Confusion Matrix:

[[24 2]

[1 13]]



Result:

Thus, the Naïve Bayes algorithm was successfully implemented, and the confusion matrix and accuracy were displayed.

15) Iris Flower Classification Using Naïve Bayes Classifier

Aim:

To implement Iris Flower Classification using the Naïve Bayes classifier in Python.

Procedure:

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing data.
4. Train the Gaussian Naïve Bayes classifier.
5. Predict the flower species.
6. Display the accuracy and confusion matrix.

Code:

```
# Experiment 2: Iris Flower Classification

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# Create and train model
model = GaussianNB()
model.fit(X_train, y_train)

# Predict flower species
y_pred = model.predict(X_test)

# Display predictions
print("Predicted Flower Species:")
for i in y_pred:
    print(iris.target_names[i])

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names,
            cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Iris Confusion Matrix")
```

```
plt.show()
```

Output:

Predicted Flower Species:

versicolor

setosa

virginica

versicolor

versicolor

setosa

versicolor

virginica

versicolor

versicolor

virginica

setosa

setosa

setosa

setosa

versicolor

virginica

versicolor

versicolor

virginica

setosa

virginica

setosa

virginica

virginica

virginica

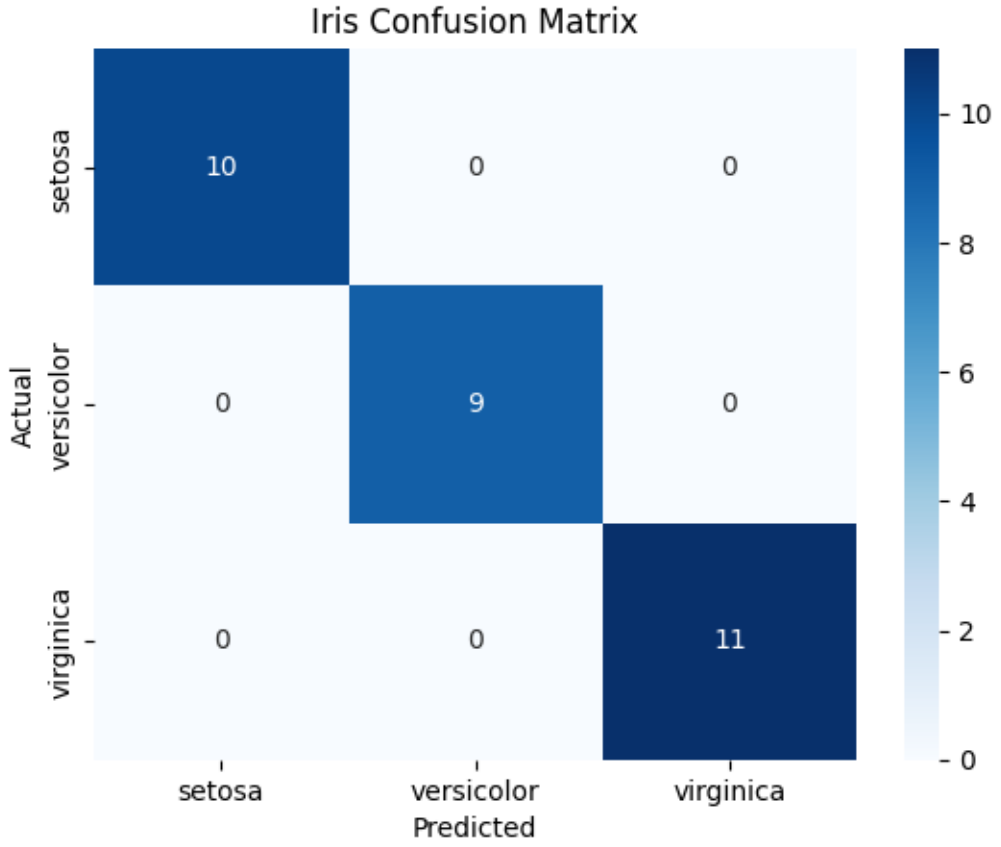
virginica

virginica

setosa

setosa

Accuracy: 1.0



Result:

Thus, Iris Flower Classification was successfully implemented using the Naïve Bayes classifier, and the accuracy and confusion matrix were displayed.

19)Bank Loan Prediction Using Naïve Bayes Classification

Aim:

To implement Naïve Bayes classification for predicting whether a bank loan will be approved or rejected.

Procedure:

1. Import the required libraries.
2. Create a bank loan dataset.
3. Separate input and output variables.
4. Split the dataset into training and testing data.
5. Train the Naïve Bayes classifier.
6. Predict loan approval and display accuracy.

Code:

```
# Experiment 3: Bank Loan Prediction


import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix


# Create dataset
data = {
    'Income': [50000, 30000, 60000, 25000, 70000,
               40000, 80000, 20000, 55000, 35000,
               90000, 28000, 65000, 45000, 75000],
    'Credit_Score': [750, 600, 800, 550, 780,
                     650, 820, 500, 720, 620,
                     850, 580, 790, 680, 810],
    'Loan_Amount': [20000, 30000, 25000, 35000, 20000,
                    30000, 25000, 40000, 22000, 32000,
```

```
                20000, 38000, 25000, 28000, 22000],
    'Approved': [1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
                 1, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

# Input and output
X = df[['Income', 'Credit_Score', 'Loan_Amount']]
y = df['Approved']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Predictions:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Predict a new loan application
new_data = pd.DataFrame(
```

```
    [[60000, 780, 25000]],
    columns=['Income', 'Credit_Score', 'Loan_Amount']
)

result = model.predict(new_data)

if result[0] == 1:
    print("Loan Approved")
else:
    print("Loan Rejected")
```

Output:

Predictions: [0 0 1]

Accuracy: 1.0

Confusion Matrix:

[[2 0]

[0 1]]

Loan Approved

Result:

Thus, the Naïve Bayes classification algorithm was successfully implemented to predict bank loan approval.

20) Future Sales Prediction Using Machine Learning

Aim:

To implement future sales prediction using the Linear Regression machine learning algorithm in Python.

Procedure:

1. Import the required libraries.
2. Create a sales dataset.
3. Separate months and sales values.
4. Train the Linear Regression model.
5. Predict future sales.
6. Display the predicted sales.

Code:

```
# Experiment 4: Future Sales Prediction


import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression


# Create sales dataset

data = {

   'Month': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

   'Sales': [100, 120, 130, 150, 170,

            180, 200, 220, 240, 260]

}


df = pd.DataFrame(data)


# Input and output

X = df[['Month']]
```

```
y = df['Sales']

# Train Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Predict future sales
future_months = pd.DataFrame(
    {'Month': [11, 12, 13]}
)

predictions = model.predict(future_months)

# Display results
print("Future Sales Prediction:")

for month, sales in zip(future_months['Month'], predictions):
    print("Month", month, ":", round(sales, 2))

# Plot graph
plt.scatter(X, y, color='blue', label='Actual Sales')
plt.plot(X, model.predict(X), color='red', label='Predicted Sales')

plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Future Sales Prediction")
plt.legend()
plt.show()
```

Output:

Future Sales Prediction:

Month 11 : 273.33

Month 12 : 290.85

Month 13 : 308.36



Result:

Thus, future sales prediction was successfully implemented using the Linear Regression algorithm, and the predicted sales values were displayed.

5: Implementation of K-Nearest Neighbours (K-NN) in Python

Aim:

To implement the K-Nearest Neighbours (K-NN) classification algorithm using Python.

Procedure:

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing data.
4. Create the K-NN classifier.
5. Train the model and predict the results.
6. Display the accuracy.

Code:

```
# Experiment 5: K-Nearest Neighbours

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create KNN model
```

```
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Display results
print("Predictions:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Output:

```
Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Result:

Thus, the K-Nearest Neighbours algorithm was successfully implemented using Python, and the classification accuracy was displayed.

7: Implementation of Logistic Regression Algorithm in Python

Aim:

To implement the Logistic Regression algorithm in Python for classification.

Procedure:

1. Import the required libraries.
2. Load the Iris dataset.
3. Select two flower classes.
4. Split the dataset into training and testing data.
5. Train the Logistic Regression model.
6. Predict the results and display accuracy.

Code:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

iris = load_iris()
X = iris.data[:, :2]
y = iris.target

# Select two classes
X = X[y != 2]
y = y[y != 2]

model = LogisticRegression()
model.fit(X, y)

plt.figure(figsize=(8, 5))

plt.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm')
```

```
plt.xlabel("Sepal Length")

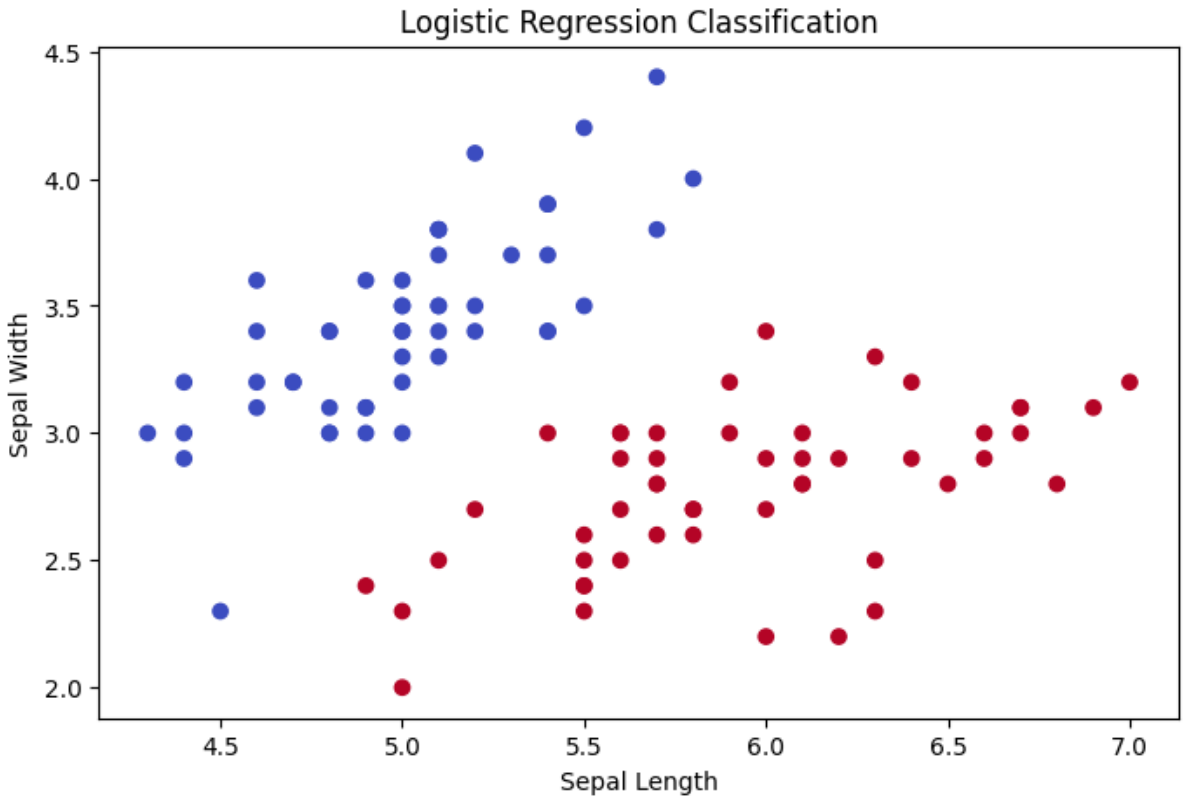
plt.ylabel("Sepal Width")

plt.title("Logistic Regression Classification")


plt.show()


print("Accuracy:", model.score(X, y))
```

Output:



Result:

Thus, the Logistic Regression algorithm was successfully implemented in Python, and the classification accuracy was displayed.

8: Implementation of Linear Regression Algorithm in Python

Aim:

To implement the Linear Regression algorithm in Python to predict values based on input data.

Procedure:

1. Import the required libraries.
2. Create a sample dataset.
3. Separate input and output variables.
4. Train the Linear Regression model.
5. Predict the output values.
6. Display the results and graph.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dataset
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 6, 8, 10])

# Train model
model = LinearRegression()
model.fit(X, y)

y_pred = model.predict(X)

print("Predicted Values:", y_pred)
print("Accuracy (R2 Score):", model.score(X, y))

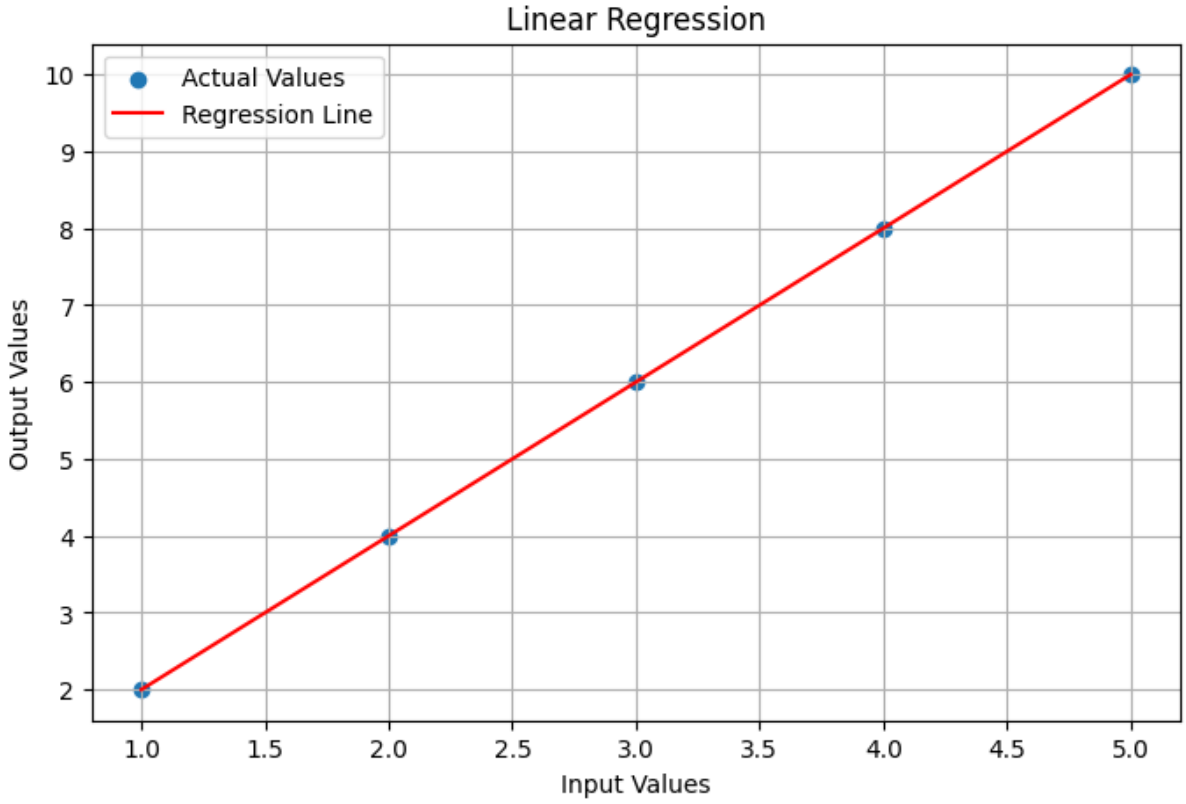
# Graph
```

```
plt.figure(figsize=(8, 5))


plt.scatter(X, y, label="Actual Values")
plt.plot(X, y_pred, color='red', label="Regression Line")


plt.xlabel("Input Values")
plt.ylabel("Output Values")
plt.title("Linear Regression")
plt.legend()
plt.grid()
plt.show()
```

Output:



Result:

Thus, the Linear Regression algorithm was successfully implemented in Python, and the predicted values were displayed.

9: Compare Linear and Polynomial Regression Using Python

Aim:

To compare Linear Regression and Polynomial Regression algorithms using Python.

Procedure:

1. Import the required libraries.
2. Create a sample dataset.
3. Apply Linear Regression.
4. Apply Polynomial Regression.
5. Predict the output values.
6. Compare both models using a graph and R^2 score.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Dataset
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([2, 5, 10, 17, 26, 37])

# Linear Regression
linear = LinearRegression()
linear.fit(X, y)

# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

polynomial = LinearRegression()
```

```
polynomial.fit(X_poly, y)

# Smooth graph values
X_curve = np.linspace(1, 6, 100).reshape(-1, 1)

linear_pred = linear.predict(X_curve)
poly_pred = polynomial.predict(poly.transform(X_curve))

# Display graph
plt.figure(figsize=(9, 5))

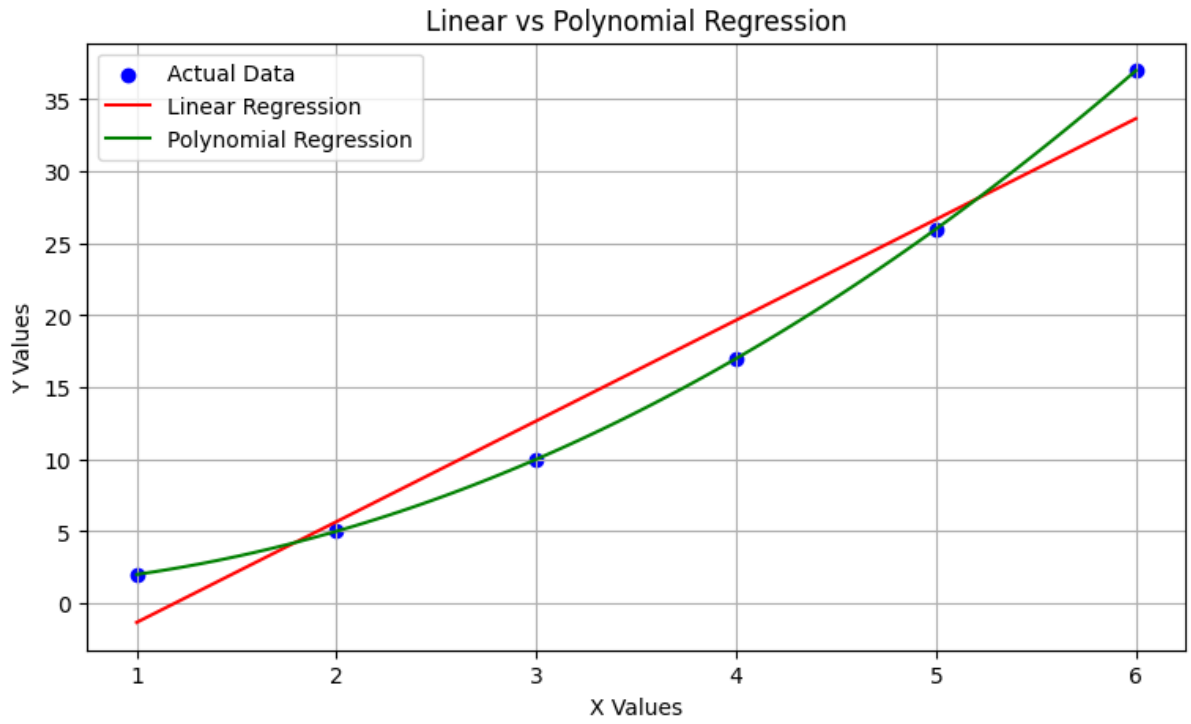
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X_curve, linear_pred, color='red',
        label='Linear Regression')

plt.plot(X_curve, poly_pred, color='green',
        label='Polynomial Regression')

plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Linear vs Polynomial Regression")
plt.legend()
plt.grid()
plt.show()

print("Linear R2 Score:", linear.score(X, y))
print("Polynomial R2 Score:",
     polynomial.score(X_poly, y))
```

Output:



Result:

Thus, Linear Regression and Polynomial Regression were successfully implemented and compared using their R^2 scores and graphs.

10: Expectation-Maximization (EM) Algorithm

Aim:

To implement the Expectation-Maximization algorithm in Python for clustering data using Gaussian Mixture Model (GMM).

Procedure:

1. Import the required libraries.
2. Generate sample dataset.
3. Apply the Gaussian Mixture Model.
4. Fit the model to the dataset.
5. Predict the clusters.
6. Display the clusters using a graph.

Code:

```
# Experiment 10: Expectation Maximization Algorithm


import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture


# Generate dataset
X, y = make_blobs(
    n_samples=200,
    centers=3,
    random_state=42
)


# Apply EM algorithm using GMM
model = GaussianMixture(
    n_components=3,
```

```
    random_state=42
)

model.fit(X)

# Predict clusters
y_pred = model.predict(X)

print("Cluster Labels:")
print(y_pred)

print("EM Algorithm Completed Successfully")

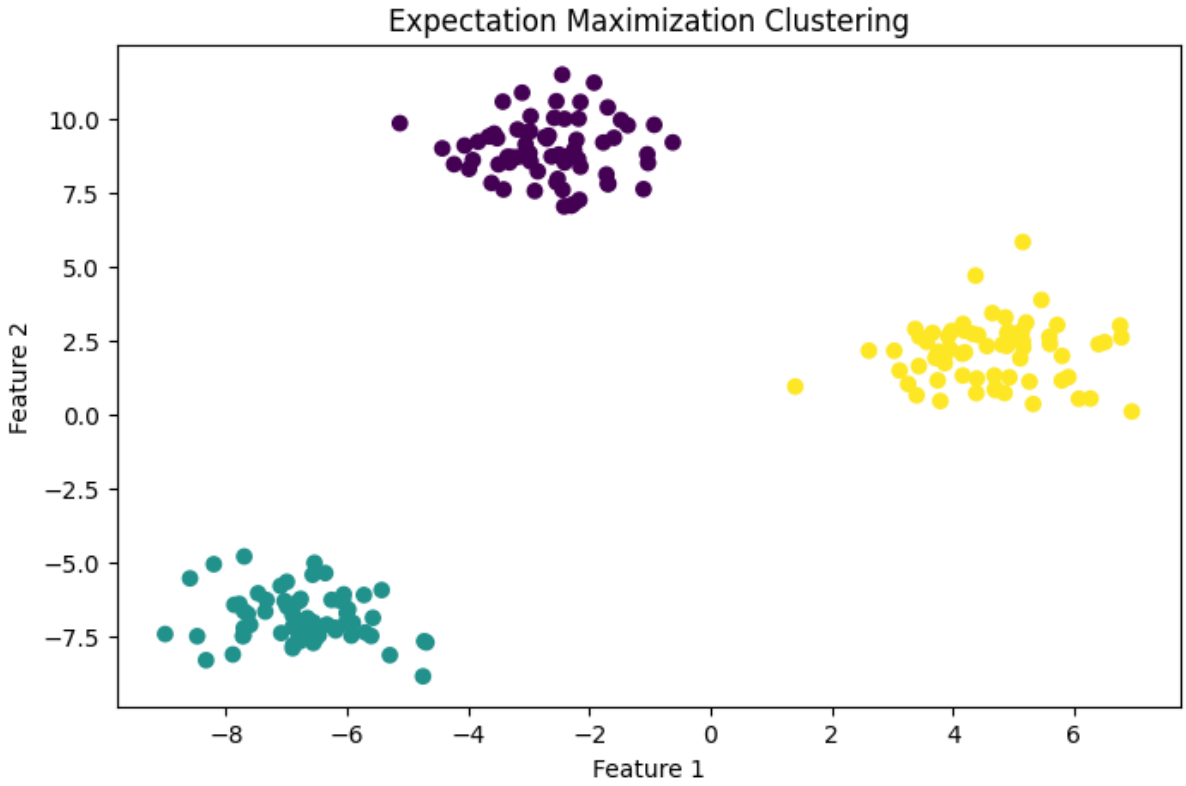
# Plot graph
plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Expectation Maximization Clustering")
plt.show()
```

Output:

```
Cluster Labels:
[2 0 1 1 1 1 0 1 2 1 2 2 2 2 2 1 0 1 2 1 2 2 1 0 1 0 0 1 0 2 2 2 1 1 0 0 1
 0 1 0 2 0 1 2 1 2 2 1 2 0 1 0 1 0 0 0 1 2 0 2 1 0 1 1 1 1 0 0 1 0 0 1 0 2
 0 2 2 0 2 0 0 2 2 1 1 1 1 0 2 1 2 0 0 0 2 1 0 2 2 0 1 1 0 1 0 2 1 2 2 1 2
 2 0 1 0 1 1 0 0 0 1 2 0 1 1 2 0 0 2 0 2 1 0 1 1 2 2 2 0 2 0 0 1 2 2 0 2 1
 1 2 0 1 2 1 2 1 1 0 2 2 0 2 2 1 0 2 2 0 1 2 1 0 2 1 0 2 0 0 0 2 0 2 1 1 1
 0 2 2 2 0 0 2 1 1 0 1 0 0 2 2]
```

EM Algorithm Completed Successfully



Result:

Thus, the Expectation-Maximization algorithm was successfully implemented using Gaussian Mixture Model, and the clusters were displayed graphically.

11: Credit Score Classification

Aim:

To implement a machine learning classification model to classify credit scores based on financial information.

Procedure:

1. Import the required libraries.
2. Create a sample credit score dataset.
3. Separate input and output variables.
4. Split the dataset into training and testing data.
5. Train the classification model.
6. Predict credit score categories and display accuracy and graph.

Code:

```
# Experiment 11: Credit Score Classification


import pandas as pd

import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, confusion_matrix


# Create dataset

data = {

   'Income': [25000, 30000, 45000, 60000, 75000,

              20000, 50000, 80000, 35000, 90000,

              28000, 65000, 40000, 85000, 55000],

   'Debt': [80, 70, 40, 20, 10, 90, 30, 5, 60, 5,

            85, 15, 50, 8, 25],

   'Payment_History': [1, 1, 2, 3, 3, 1, 2, 3, 1, 3,
```

```
                    1, 3, 2, 3, 2],
    'Credit_Score': ['Poor', 'Poor', 'Average', 'Good',
                     'Good', 'Poor', 'Average', 'Good',
                     'Poor', 'Good', 'Poor', 'Good',
                     'Average', 'Good', 'Average']
}

df = pd.DataFrame(data)

# Input and output
X = df[['Income', 'Debt', 'Payment_History']]
y = df['Credit_Score']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Predicted Credit Scores:", y_pred)
print("Actual Credit Scores:", list(y_test))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Confusion Matrix
cm = confusion_matrix(
    y_test, y_pred,
    labels=['Poor', 'Average', 'Good']
)

plt.figure(figsize=(7, 5))
plt.imshow(cm, cmap='Blues')
plt.colorbar()

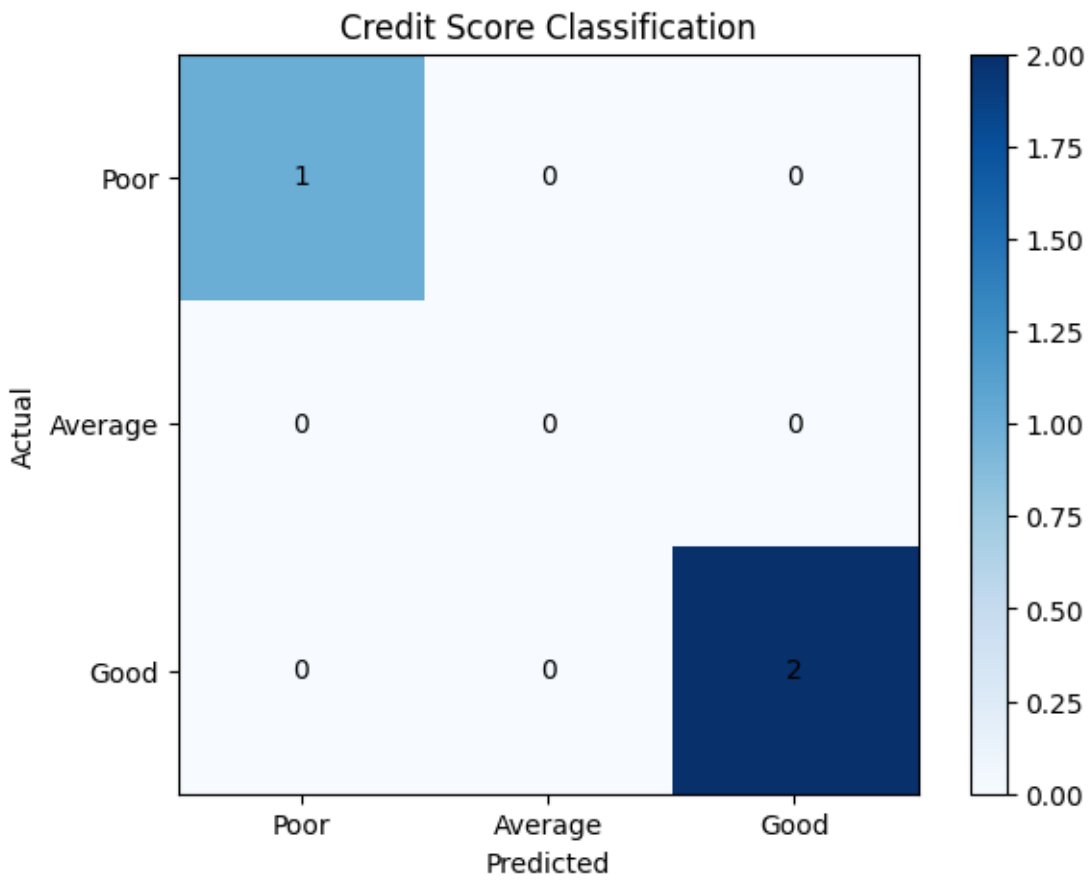
plt.xticks(range(3), ['Poor', 'Average', 'Good'])
plt.yticks(range(3), ['Poor', 'Average', 'Good'])

for i in range(3):
    for j in range(3):
        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Credit Score Classification")
plt.show()
```

Output:

```
Predicted Credit Scores: ['Good' 'Good' 'Poor']
Actual Credit Scores: ['Good', 'Good', 'Poor']
Accuracy: 1.0
```



Result:

Thus, the Credit Score Classification model was successfully implemented using Decision Tree Classifier, and the predicted results and confusion matrix were displayed.

12: Iris Flower Classification Using KNN

Aim:

To implement Iris Flower Classification using the K-Nearest Neighbours (KNN) algorithm in Python.

Procedure:

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing data.
4. Create the KNN classifier.
5. Train the model and predict flower species.
6. Display accuracy and confusion matrix with a graph.

Code:

```
# Experiment 12: Iris Flower Classification using KNN


import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix


# Load dataset

iris = load_iris()

X = iris.data

y = iris.target


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Display results
print("Predicted Flower Names:")

for i in y_pred:
    print(iris.target_names[i])

print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
```

```
    yticklabels=iris.target_names
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Iris Flower Classification using KNN")
plt.show()
```

Output:

Predicted Flower Names:

versicolor

setosa

virginica

versicolor

versicolor

setosa

versicolor

virginica

versicolor

versicolor

virginica

setosa

setosa

setosa

setosa

versicolor

virginica

versicolor

versicolor

virginica

setosa

virginica

setosa

virginica

virginica

virginica

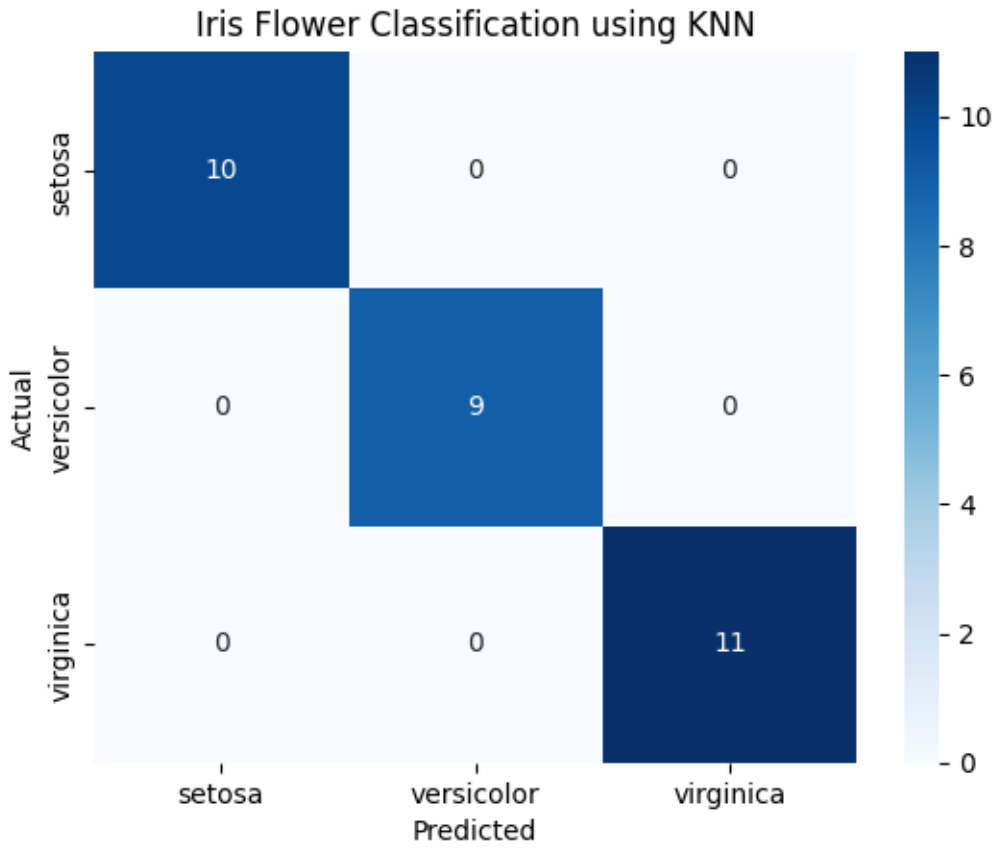
virginica

virginica

setosa

setosa

Accuracy: 1.0



Result:

Thus, Iris Flower Classification was successfully implemented using the K-Nearest Neighbours algorithm, and the accuracy and confusion matrix were displayed.

13: Car Price Prediction Model Using Python

Aim:

To implement a Car Price Prediction model using the Linear Regression machine learning algorithm in Python.

Procedure:

1. Import the required libraries.
2. Create a sample car dataset.
3. Separate input and output variables.
4. Split the dataset into training and testing data.
5. Train the Linear Regression model.
6. Predict car prices and display the graph.

Code:

```
# Experiment 13: Car Price Prediction


import pandas as pd
import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score


# Create dataset
data = {
    'Year': [2015, 2016, 2017, 2018, 2019,
             2020, 2021, 2022, 2023, 2024],
    'Kilometers': [80000, 70000, 65000, 60000, 50000,
                   40000, 35000, 30000, 20000, 10000],
    'Price': [400000, 450000, 500000, 550000, 600000,
              650000, 700000, 750000, 850000, 950000]
```

```
}

df = pd.DataFrame(data)

# Input and output
X = df[['Year', 'Kilometers']]
y = df['Price']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Actual Prices:", list(y_test))
print("Predicted Prices:", y_pred.round(2))
print("R2 Score:", r2_score(y_test, y_pred))

# Predict new car price
new_car = pd.DataFrame(
    [[2022, 25000]],
    columns=['Year', 'Kilometers']
)
```

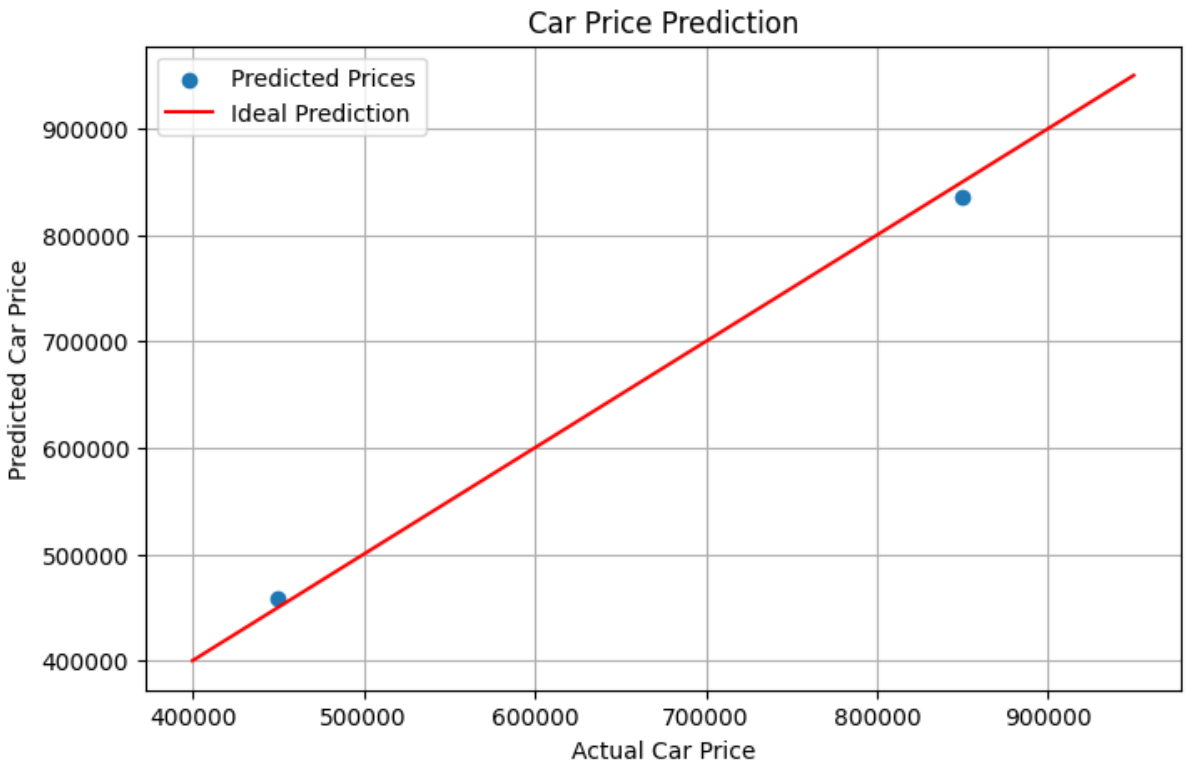
```
price = model.predict(new_car)
print("Predicted Car Price:", round(price[0], 2))

# Graph
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred, label="Predicted Prices")
plt.plot([y.min(), y.max()], [y.min(), y.max()],
        color='red', label="Ideal Prediction")

plt.xlabel("Actual Car Price")
plt.ylabel("Predicted Car Price")
plt.title("Car Price Prediction")
plt.legend()
plt.grid()
plt.show()
```

Output:

```
Actual Prices: [850000, 450000]
Predicted Prices: [835185.19 458333.33]
R2 Score: 0.9963884602194788
Predicted Car Price: 797222.22
```



Result:

Thus, the Car Price Prediction model was successfully implemented using Linear Regression, and the predicted car prices were displayed graphically.

14: House Price Prediction Using Machine Learning

Aim:

To implement House Price Prediction using the Linear Regression machine learning algorithm in Python.

Procedure:

1. Import the required libraries.
2. Create a sample house dataset.
3. Separate input and output variables.
4. Split the dataset into training and testing data.
5. Train the Linear Regression model.
6. Predict house prices and display the graph.

Code:

```
# Experiment 14: House Price Prediction


import pandas as pd

import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, r2_score


# Create dataset

data = {

    'Area': [1000, 1200, 1500, 1800, 2000,

             2200, 2500, 2800, 3000, 3500],

    'Bedrooms': [2, 2, 3, 3, 3, 4, 4, 4, 5, 5],

    'Age': [10, 8, 7, 6, 5, 4, 3, 2, 2, 1],

    'Price': [2000000, 2500000, 3000000, 3500000,

              4000000, 4500000, 5000000, 5500000,
```

```
            6000000, 7000000]
}

df = pd.DataFrame(data)

# Input and output
X = df[['Area', 'Bedrooms', 'Age']]
y = df['Price']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Actual House Prices:", list(y_test))
print("Predicted House Prices:", y_pred.round(2))
print("R2 Score:", r2_score(y_test, y_pred))

# Predict new house price
new_house = pd.DataFrame(
    [[2000, 3, 5]],
    columns=['Area', 'Bedrooms', 'Age']
```

```
)

price = model.predict(new_house)
print("Predicted House Price:", round(price[0], 2))

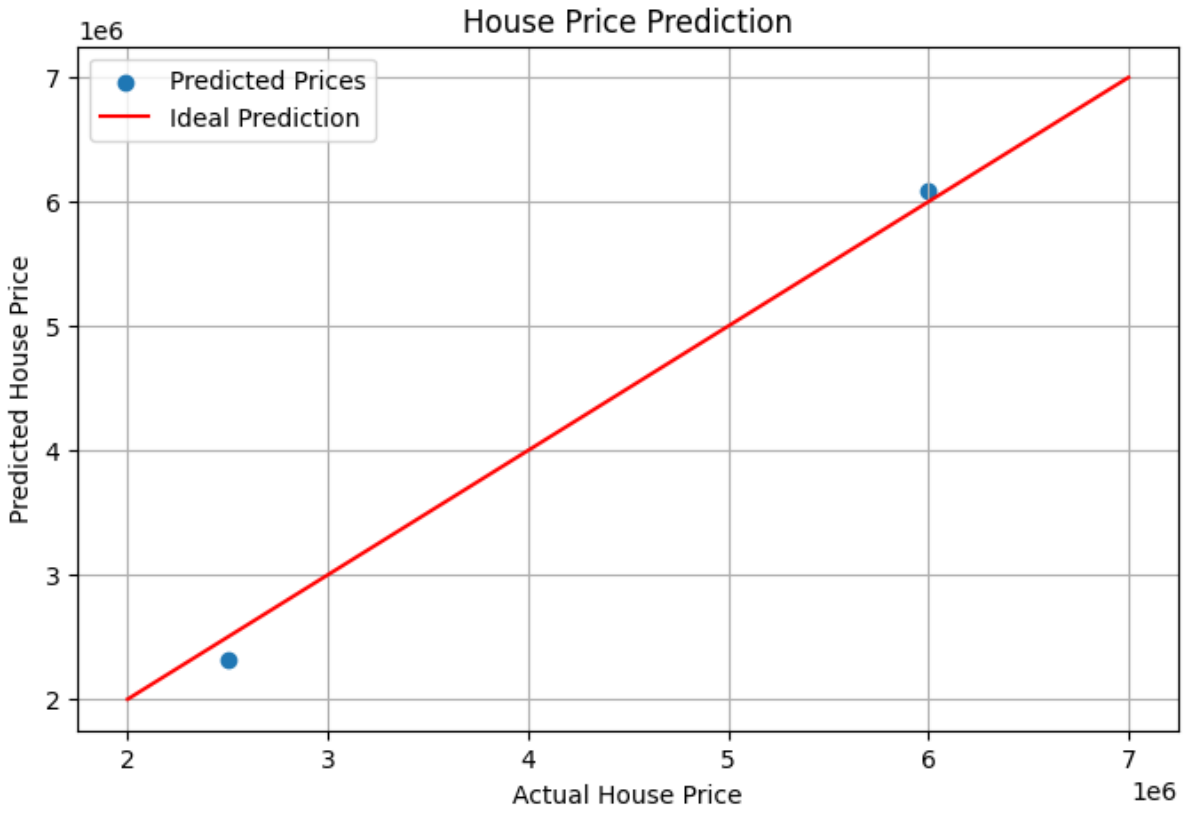
# Graph
plt.figure(figsize=(8, 5))

plt.scatter(y_test, y_pred, label="Predicted Prices")
plt.plot([y.min(), y.max()], [y.min(), y.max()],
        color='red', label="Ideal Prediction")

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("House Price Prediction")
plt.legend()
plt.grid()
plt.show()
```

Output:

```
Actual House Prices: [6000000, 2500000]
Predicted House Prices: [6083212.44 2318134.72]
R2 Score: 0.9934695034683838
Predicted House Price: 3927668.39
```



Result:

Thus, the House Price Prediction model was successfully implemented using Linear Regression, and the predicted house prices were displayed graphically.